

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	07/29/2026	DATE SUBMITTED:	07/29/2026
NAME:	Jiro Luis A. Katindig	INSTRUCTOR:	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 Linked Lists

1. PROCEDURE OUTPUT

Screenshot

```

1 #include <iostream>
2
3 class Node {
4 public:
5     char data;
6     Node *next;
7 };
8
9 int main(){
10
11     //creation of nodes
12     Node* head = nullptr;
13     Node* second = nullptr;
14     Node* third = nullptr;
15     Node* fourth = nullptr;
16     Node* fifth = nullptr;
17     Node* sixth = nullptr;
18     Node* seventh = nullptr;
19
20     //allocate 6 nodes in the heap
21     head = new Node;
22     second = new Node;
23     third = new Node;
24     fourth = new Node;
25     fifth = new Node;
26     sixth = new Node;
27
28     //storing data in nodes
29     head->data = 'C';
30     second->data = 'P';
31     third->data = 'E';
32     fourth->data = 'O';
33     fifth->data = 'I';
34     sixth->data = 'O';
35
36     //linking nodes
37     head->next = second;
38     second->next = third;
39     third->next = fourth;
40     fourth->next = fifth;
41     fifth->next = sixth;
42     sixth->next = nullptr;
43
44     //traversal of the linked list
45     Node *curr = head;
46     while (curr != nullptr) {
47         std::cout << curr->data;
48         curr = curr->next;
49     }
50     std::cout << std::endl;
51     return 0;
52 }

```

```

#include <iostream>
#include "singly_ll.h"

int main(){

    //creation of nodes for JIRO
    SingleList<char>* head = nullptr;
    SingleList<char>* second = nullptr;
    SingleList<char>* third = nullptr;
    SingleList<char>* fourth = nullptr;
    SingleList<char>* fifth = nullptr;
    SingleList<char>* sixth = nullptr;
    SingleList<char>* seventh = nullptr;

    //allocate 4 nodes in the heap
    head = new SingleList<char>;
    second = new SingleList<char>;
    third = new SingleList<char>;
    fourth = new SingleList<char>;
    fifth = new SingleList<char>;
    sixth = new SingleList<char>;
    seventh = new SingleList<char>;

    //storing data in nodes
    head->data = 'D';
    second->data = 'E';
    third->data = 'N';
    fourth->data = 'I';
    fifth->data = 'S';
    sixth->data = 'E';
    seventh->data = 'E';

    //linking nodes
    head->next = second;
    second->next = third;
    third->next = fourth;
    fourth->next = fifth;
    fifth->next = sixth;
    sixth->next = seventh;
    seventh->next = nullptr;

    std::cout << "Testing of traversal: \n";
    ListTraversal(head);

    std::cout << "Testing of sllInsertHead: \n";
    sllInsert('I', &head);
    ListTraversal(head);

    std::cout << "Testing of sllGeneralInsert: \n";
    sllGeneralInsert('X', head->next->next);
    ListTraversal(head);

    std::cout << "Testing of sllInsertEnd: \n";
    sllInsertEnd('U', &head);
    ListTraversal(head);

    std::cout << "Testing of deleting the node: \n";
    sllDeleteNode('U', &head);
    ListTraversal(head);
    sllDeleteNode('Q', &head);
    ListTraversal(head);
    sllDeleteNode('X', &head);
    ListTraversal(head);

    std::cout << "deleting all nodes \n";
    sllDeleteList(&head);
    ListTraversal(head);

    return 0;
}

```

Discussion

When I first ran the "CPE010" code, the terminal was completely blank because there was no code to print anything, so I had to add a traversal loop myself just to see the output. For the second code, I followed sir's lesson using singly_ll.h. I tested a 7-letter name ("DENNISE") and used the required functions like sllinsert and sllDeleteNode instead of manually linking every node by hand.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	<pre> template <typename T> void ListTraversal(SingleList<T>* head) { while(head != nullptr) { //print data of n std::cout << head->data; if(head->next != nullptr) { std::cout << " -> "; } // go to the next node head = head->next; } std::cout << std::endl; } </pre>
Insertion at head	<pre> template <typename T> void sllinsert(T newData, SingleList<T>** currenthead) { // 1. Allocate memory for new node SingleList<T>* newNode = new SingleList<T>; // 2. Put in the data newNode->data = newData; // 3. Set Next of the new node to point to the previous head newNode->next = *currenthead; // 4. Reset Head to point to the new node *currenthead = newNode; } </pre>
Insertion at any part of the list	<pre> template <typename T> void sllGeneralInsert(T newData, SingleList<T>* previousNode) { //1. Check if its the head node or not if(previousNode == nullptr) { //2. If null, print "previous node cannot be null" std::cout << "previous node cannot be null" << std::endl; return; } //3. Allocate a new node SingleList<T>* newNode = new SingleList<T>; //4. Store data in the new node newNode->data = newData; //5. Point new node to the node previous node was pointing to newNode->next = previousNode->next; //6. Point previous node to the new node previousNode->next = newNode; } </pre>
Insertion at the end	<pre> template <typename T> void sllinsertEnd(T newData, SingleList<T>** head) { //1. Allocate a new node SingleList<T>* newNode = new SingleList<T>; //2. Dereference to the head node SingleList<T>* currentNode = *head; //3. Store data in new node newNode->data = newData; newNode->next = nullptr; //4. Point to Null //5. Traverse the list until next of the node is null while(currentNode->next != nullptr){ currentNode = currentNode->next; } // 6. Point the next of the current node to the new node currentNode->next = newNode; } </pre>
Deletion of a node	<pre> template <typename T> void sllDeleteNode(T findData, SingleList<T>** head){ //if there is nothing to be delete? if(*head == nullptr) return; SingleList<T>* currNode = *head; SingleList<T>* prevNode = nullptr; //searching the nodes: 1.the node doesnt exist + we found data while(currNode != nullptr && currNode->data != findData) { prevNode = currNode; currNode = currNode->next; } //if data not found currNode == nullptr in the while loop: if(currNode == nullptr) { std::cout << "the data is not found \n" << std::endl; return; } //if data was found: if(prevNode == nullptr){ *head = currNode->next; }else{ prevNode->next = currNode->next; } delete currNode; } </pre>

Table 3-2. Code for the List Operations

a.	Source Code	<pre>std::cout<< "Testing of traversal: \n"; ListTraversal(head);</pre> Figure 1. Code Snippet of Testing the function ListTraversal
	Console	<pre>Testing of traversal: D -> E -> N -> N -> I -> S -> E</pre> Figure 2. Snippet of the Output in the console from ListTraversal
b.	Source Code	<pre>std::cout<< "Testing of sllInsertHead: \n"; sllinsert('J', &head); ListTraversal(head);</pre> Figure 3. Code Snippet of Testing the function sll_insert_head
	Console	<pre>Testing of sllInsertHead: J -> D -> E -> N -> N -> I -> S -> E</pre> Figure 4. Snippet of the Output inside the console from sll_insert_head
c.	Source Code	<pre>std::cout<< "Testing of sllGeneralInsert: \n"; sllGeneralInsert('X', head->next->next); ListTraversal(head);</pre> Figure 5. Code Snippet of the sll_general_insert
	Console	<pre>Testing of sllGeneralInsert: J -> D -> E -> X -> N -> N -> I -> S -> E</pre> Figure 6. Snippet of the Output inside the console from sll_general_insert
d.	Source Code	<pre>std::cout<< "Testing of sllInsertEnd: \n"; sllinsertEnd('U', &head); ListTraversal(head);</pre> Figure 7. Code Snippet of Testing the function sllInsertEnd
	Console	<pre>Testing of sllInsertEnd: J -> D -> E -> X -> N -> N -> I -> S -> E -> U</pre> Figure 8. Snippet of the Output inside the console from sllInsertEnd
e.	Source Code	<pre>std::cout<< "Testing of deleting the node: \n"; sllDeleteNode('U', &head); ListTraversal(head); sllDeleteNode('Q', &head); ListTraversal(head); sllDeleteNode('X', &head); ListTraversal(head);</pre> Figure 9. Code Snippet of Testing the function sllDeleteNode
	Console	<pre>Testing of deleting the node: J -> D -> E -> X -> N -> N -> I -> S -> E</pre> Figure 10. Snippet of the Output inside the console from sllDeleteNode
f.	Source Code	<pre>std::cout<<"deleting all nodes \n"; sllDeleteList(&head); ListTraversal(head);</pre> Figure 11. Code Snippet of sllDeleteList on main.cpp
	Console	<pre>J -> D -> E -> X -> N -> N -> I -> S -> E J -> D -> E -> N -> N -> I -> S -> E deleting all nodes</pre> Figure 12. Snippet of the Output inside the console from sllDeleteList

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshots(s)	Analysis
<pre> template <typename T> void dllTraverse(DoubleList<T>* currentNode){ //temp variable to store the last node visited DoubleList<T> *tail; //check if list is empty if(currentNode == nullptr){ std::cout<<"The list is empty"<<std::endl; return; } //move forward std::cout<<"Forward: \n"; while(currentNode != nullptr){ std::cout<< currentNode->data << " "; tail = currentNode; currentNode = currentNode->next; } // add a next line std::cout<<std::endl; //move backward std::cout<<"Backward: \n"; while(tail != nullptr){ std::cout<<tail->data<<" "; tail = tail->prev; } } </pre>	This function prints the list in both directions. It loops forward through the nodes using next and saves the last node as tail. Then, it uses tail to loop backward using prev so it can print the list in reverse.
<pre> //inserting at the head of the list template <typename T> void dllInsertHead(T newData, DoubleList<T> ** currentHead){ //creating a new node DoubleList<T>* newNode = CreateNewNode(newData); //new node should point to the current head newNode->next = *currentHead; //current Head should point back to the newNode (*currentHead)->prev = newNode; //update the pointer head *currentHead = newNode; } </pre>	This function adds a new node at the very beginning of the list. It links the new node's next to the old head, sets the old head's prev back to the new node, and then updates head to point to the new node.
<pre> template <typename T> void dllInsertEnd(T newData, DoubleList<T>* currentHead){ //create a new node DoubleList<T>* newNode = CreateNewNode(newData); //traverse until we reach the last node while(currentHead->next != nullptr){ currentHead = currentHead->next; } //connect the last node to the new node currentHead->next = newNode; //connect the new node back to last node newNode->prev = currentHead; } </pre>	This function attaches a new node to the end of the list. It loops until it reaches the last node, connects the last node's next to the new node, and points the new node's prev back to that last node.

```
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode) {
    if (prevNode == nullptr){
        std::cout << "Prevnode cannot be null" << std::endl;
        return;
    }

    DoubleList<T>* newNode = CreateNewNode(newData);

    //point new node next to prev node next
    newNode->next = prevNode->next;

    //point new node next to prev node
    newNode->prev = prevNode;

    //if prevnode isnt the last node, adjust the next node's prev pointer
    if (prevNode->next != nullptr){
        prevNode->next->prev = newNode;
    }

    //point prevnode next to the new node
    prevNode->next = newNode;
}
}
```

This inserts a new node right after a specific node. It checks if the previous node exists, creates the new node, and hooks up both the next and prev pointers on both sides so the new node sits safely in the middle.

```
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>*& head) {
    if (*head == nullptr) {
        std::cout << "List is empty." << std::endl;
        return;
    }

    DoubleList<T>* currNode = *head;

    //search for the node
    while(currNode != nullptr && currNode->data != findData) {
        currNode = currNode->next;
    }

    //data not found
    if (currNode == nullptr) {
        std::cout << "Data '" << findData << "' not found in list.\n";
        return;
    }

    //if node to delete is the head
    if (*head == currNode) {
        *head = currNode->next;
    }

    //adjust next node's prev pointer if not deleting tail
    if (currNode->next != nullptr) {
        currNode->next->prev = currNode->prev;
    }

    //adjust previous node's next pointer if not deleting head
    if (currNode->prev != nullptr) {
        currNode->prev->next = currNode->next;
    }

    delete currNode;
}
}
```

This searches for a node with matching data and deletes it. It reconnects the pointers of the neighboring nodes around it so they skip the target node, then frees its memory using delete.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Main.cpp

```
#include <iostream>
#include <string>
#include "appleMusic_ll.h"

// prints a section header so each step of the demo stands out
void printHeader(const std::string& title) {
    std::string border(title.length() + 4, "=");
    std::cout << "\n" << border << "\n";
    std::cout << " " << title << "\n";
    std::cout << border << "\n";
}

int main() {
    // create queue
    printHeader("CREATE QUEUE");
    CircularList<Song>* queue = nullptr;
    addSong(Song("Summer Crush", "HELLMERRY"), &queue);
    addSong(Song("Halik Sobrang Diin Pt. 2", "Gat Putch"), &queue);
    addSong(Song("Original", "Uncle Dags"), &queue);
    addSong(Song("Chinay", "La Mave"), &queue);
    std::cout << "Queue created. Playing all songs:\n";
    playAllSongs(queue);

    // add song
    printHeader("ADD SONG");
    std::cout << "Adding 'MYSB (Miss You So Bad)' by Skusta Clee...\n";
    addSong(Song("MYSB (Miss You So Bad)", "Skusta Clee"), &queue);
    playAllSongs(queue);

    // remove song
    printHeader("REMOVE SONG");
    std::cout << "Removing 'Chinay' by La Mave...\n";
    removeSong(Song("Chinay", "La Mave"), &queue);
    playAllSongs(queue);

    // loop
    printHeader("PLAY ALL (LOOP)");
    playAllSongs(queue);
```

```
// next / previous
printHeader("NEXT / PREVIOUS");
CircularList<Song>* nowPlaying = queue;
std::cout << "Now playing : " << nowPlaying->data << std::endl;
nextSong(&nowPlaying);
std::cout << "Next : " << nowPlaying->data << std::endl;
nextSong(&nowPlaying);
std::cout << "Next : " << nowPlaying->data << std::endl;
previousSong(&nowPlaying);
std::cout << "Previous : " << nowPlaying->data << std::endl;

// remove non-existent song
printHeader("REMOVE NON-EXISTENT SONG");
std::cout << "Attempting to remove 'Paano Ka Ba?' by Hev Abi...\n";
removeSong(Song("Paano Ka Ba?", "Hev Abi"), &queue);

// confirm the queue loops back to the start after a full cycle
printHeader("CONFIRM LOOPING");
CircularList<Song>* cursor = queue;
int songCount = 0;
CircularList<Song>* counter = queue;
do {
    songCount++;
    counter = counter->next;
} while (counter != queue);

for (int played = 0; played < songCount * 2; played++) {
    std::cout << " [" << (played % songCount) + 1 << "/" << songCount << "]" << " "
        << cursor->data << std::endl;
    nextSong(&cursor);
}
std::cout << std::endl;

return 0;
```

Applemusic_ll.h

```
#ifndef APPLEMUSIC_LL_H
#define APPLEMUSIC_LL_H

#include <iostream>
#include <string>

// A song stored in the Apple Music queue
struct Song {
    std::string title;
    std::string artist;

    bool operator==(const Song& other) const {
        return title == other.title && artist == other.artist;
    }
};

std::ostream& operator<<(std::ostream& os, const Song& s) {
    os << s.title << " - " << s.artist;
    return os;
}

// node creation
template <typename S>
class CircularList {
public:
    S data;
    CircularList<S>* next = nullptr;
    CircularList<S>* prev = nullptr;
};

// add song at the end of the Apple Music queue
template <typename S>
void addSong(S newSong, CircularList<S>*& head) {
    CircularList<S>* newNode = new CircularList<S>;
    newNode->data = newSong;

    // empty queue
    if (*head == nullptr) {
        newNode->next = newNode;
        newNode->prev = newNode;
        *head = newNode;
        return;
    }

    // linking
    CircularList<S>* tail = (*head)->prev;
    tail->next = newNode;
    newNode->prev = tail;
    newNode->next = *head;
    (*head)->prev = newNode;
}

// remove a song
template <typename S>
void removeSong(S findSong, CircularList<S>*& head) {
    if (*head == nullptr) return;

    CircularList<S>* currNode = *head;

    do {
        if (currNode->data == findSong) {
            if (currNode->next == currNode) {
                *head = nullptr;
            } else {
                currNode->prev->next = currNode->next;
                currNode->next->prev = currNode->prev;
                if (currNode == *head) {
                    *head = currNode->next;
                }
            }
            delete currNode;
            return;
        }
        currNode = currNode->next;
    } while (currNode != *head);

    std::cout << "\"" << findSong << "\" was not found in the queue\n";
}

// play every song once
template <typename S>
void playAllSongs(CircularList<S>*& head) {
    if (head == nullptr) {
        std::cout << "the queue was empty\n";
        return;
    }

    CircularList<S>* currNode = head;
    do {
        std::cout << currNode->data;
        currNode = currNode->next;
        if (currNode != head) std::cout << " -> ";
    } while (currNode != head);
    std::cout << std::endl;
}

template <typename S>
void nextSong(CircularList<S>*& current) {
    if (*current == nullptr) return;
    *current = (*current)->next;
}

template <typename S>
void previousSong(CircularList<S>*& current) {
    if (*current == nullptr) return;
    *current = (*current)->prev;
}

#endif
```

Output

```
=====
CREATE QUEUE
=====
Queue created. Playing all songs:
Summer Crush - HELLMERRY -> Halik Sobrang Diin Pt. 2 - Gat Putch -> Original - Uncle Dags
-> Chinay - La Mave

=====
ADD SONG
=====
Adding 'MYSB (Miss You So Bad)' by Skusta Clee...
Summer Crush - HELLMERRY -> Halik Sobrang Diin Pt. 2 - Gat Putch -> Original - Uncle Dags
-> Chinay - La Mave -> MYSB (Miss You So Bad) - Skusta Clee

=====
REMOVE SONG
=====
Removing 'Chinay' by La Mave...
Summer Crush - HELLMERRY -> Halik Sobrang Diin Pt. 2 - Gat Putch -> Original - Uncle Dags
-> MYSB (Miss You So Bad) - Skusta Clee

=====
PLAY ALL (LOOP)
=====
Summer Crush - HELLMERRY -> Halik Sobrang Diin Pt. 2 - Gat Putch -> Original - Uncle Dags
-> MYSB (Miss You So Bad) - Skusta Clee

=====
NEXT / PREVIOUS
=====
Now playing : Summer Crush - HELLMERRY
Next       : Halik Sobrang Diin Pt. 2 - Gat Putch
Next       : Original - Uncle Dags
Previous   : Halik Sobrang Diin Pt. 2 - Gat Putch

=====
REMOVE NON-EXISTENT SONG
=====
Attempting to remove 'Paano Ka Ba?' by Hev Abi...
"Paano Ka Ba? - Hev Abi" was not found in the queue

=====
CONFIRM LOOPING
=====
[1/4] Summer Crush - HELLMERRY
[2/4] Halik Sobrang Diin Pt. 2 - Gat Putch
[3/4] Original - Uncle Dags
[4/4] MYSB (Miss You So Bad) - Skusta Clee
[1/4] Summer Crush - HELLMERRY
[2/4] Halik Sobrang Diin Pt. 2 - Gat Putch
[3/4] Original - Uncle Dags
[4/4] MYSB (Miss You So Bad) - Skusta Clee
```

Summary of Lessons Learned

In this lab, I learned how singly, doubly, and circular linked lists work in C++. I saw how nodes use pointers (next and prev) to connect data dynamically in memory instead of using fixed array sizes. I also learned that adding a prev pointer lets us walk backward through a list, while connecting the last node back to the head creates a circular list that never hits a nullptr.

Analysis of the Procedure

We started by manually setting up nodes line by line, which quickly became repetitive and inefficient. To make things easy for us, we built reusable functions for traversal, insertion, and deletion. Going through this step by step showed me how important it is to relink pointers in the right order so we don't accidentally break the chain or cause memory leaks.

Analysis of the Supplementary Activity

For the music player problem, I created a circular doubly linked list to simulate an Apple Music queue. This structure was perfect for a playlist because pressing "next" or "previous" moves smoothly through songs, and reaching the last song automatically loops back to the first. Testing functions like adding tracks, deleting tracks (and checking what happens with non-existent songs), and looping through the playlist proved that the pointers were linked correctly.

Concluding Statement / Feedback

I feel like I did well in this activity because all my code compiled without errors and produced the expected console outputs. My main area for improvement is getting better at handling edge cases, like making sure I always check for empty lists or head node deletions so the program never crashes with a segmentation fault.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.